Workflow Lisp Frontend

Full Drain vs. Ralph Loop

Implementation quality, throughput, churn, and DSL semantics

PL frontend	Performance	Next semantics
Parsing, ASTs, type checks, macro/module semantics, and lowering into the existing workflow runtime.	Static comparison of full workflow drain coverage against the faster Ralph iteration loop.	Adjudicated providers as a step toward workflow-native judging and search loops.

Workflow DSL vocabulary

Workflow

A named graph of work. It declares shared context, available providers, and ordered or routed steps.

Step

One executable node in the workflow. A step may run a provider, command, assertion, call, loop, branch, or other DSL operation.

Provider

A reusable command template for an external worker, such as `codex exec`. Steps bind to providers and pass them prompts, files, and params.

In the Ralph loop

`Ralph` is the step. `codex` is the provider. The workflow routes successful Ralph executions back to Ralph again.

Ralph resolves to one Codex command

```yaml
context:
  workflow_model: gpt-5.4
  workflow_effort: high

providers:
  codex:
    command:
      - codex
      - exec
      - --model
      - "${model}"
      - --config
      - "reasoning_effort=${reasoning_effort}"
    input_mode: stdin
    defaults:
      model: "${context.workflow_model}"
      reasoning_effort: "${context.workflow_effort}"
```

What this YAML does

- `provider: codex` selects this command template.
- `context` fills in model and reasoning effort.
- The composed prompt is sent to the child process on `stdin`.

Resolved command shape

```
codex exec ... -model gpt-5.4
-config reasoning_effort=high
```

The step validates docs and injects paths

```yaml
- name: Ralph
  provider: codex
  depends_on:
    required:
      - docs/design/workflow_lisp_frontend_specification.md
      - docs/design/workflow_lisp_frontend_mvp_specification.md
    inject:
      mode: list
      position: prepend
      instruction: "Use these design documents as the contract
       for this iteration:"
  input_file: workflows/examples/prompts/ralph_lisp_forever/ralph
      .md
```

Runtime dependency contract

- Both docs must exist before the step runs.
- Paths resolve relative to the workspace.
- `mode: list` injects filenames only.
- The docs are not copied into the prompt.
- Codex can read the files on demand from the checkout.

Small prompt, unbounded loop

Final stdin prompt

```
Use these design documents as the contract for this iteration:
  - docs/design/workflow_lisp_frontend_mvp_specification.md
  - docs/design/workflow_lisp_frontend_specification.md

review the lisp mvp and full design docs in context of the progress on
      its implementation in this repository. Select a non-implemented
      section of suitable size and draft an implementation plan for it.
      Then, execute the plan.
```

```
on:
  success:
    goto: Ralph
```

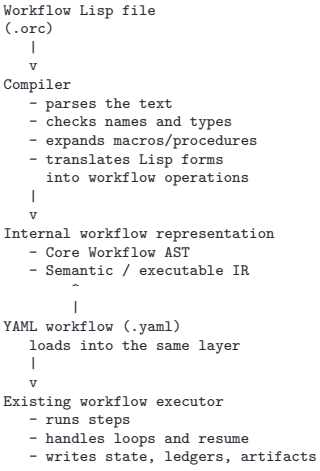
Routing

On success, `goto: Ralph` loops back to the same step with no iteration cap.

No hidden suffix

No output contract is appended: Ralph declares no `expected_outputs`, `output_bundle`, or `variant_output`.

Workflow Lisp as a PL frontend



Why not just a DAG?

Workflows are not just one-way task graphs. Steps can branch, retry, pause/resume, repair saved state, and commit outputs. Ralph is the simplest example: on success, it sends itself around again.

What the compiler produces

It does not emit machine code or rewrite the workflow as YAML. It builds checked workflow objects that the existing executor runs.

Compiler terms in plain language

Term	Meaning in this workflow frontend
Syntax forms	Parsed source shapes with enough structure to say what was written and where errors should point.
Frontend AST	The Lisp-specific tree after names and forms are understood as workflow concepts.
Typed AST	The frontend tree after the compiler checks records, unions, workflow inputs, return values, and allowed operations.
Lowering	The translation from Lisp-specific concepts into the shared workflow representation.
IR	Intermediate representation: the validated meaning the runtime can execute, resume, and persist.

Short version

The compiler turns human-authored Lisp into a checked workflow meaning. The executor then runs that meaning with the same state, ledger, artifact, and recovery machinery used by YAML workflows.

What the AST means

Authored Lisp shape

```
(defworkflow RalphDrain
  (step Ralph
    (provider codex)
    (on success Ralph)))
```

Simplified tree shape

```
Workflow
  name: RalphDrain
  steps:
    Step
      name: Ralph
      run: Provider(codex)
      on_success: Goto(Ralph)
```

Point

The AST is not text to execute directly. It is the workflow split into named parts so the compiler can check it, transform it, and lower it into the runtime's workflow representation.

Full drain vs. Ralph loop: throughput

Metric	Full Lisp drain	Ralph loop
Elapsed wall time inspected	31.45 hours	7.51 hours
Completed units	12 design gaps	95 Ralph iterations
Raw rate	0.38 design gaps/hour	12.65 iterations/hour
Durable design gaps	12 recorded gaps	0 ledger-recorded gaps
Functional MVP completion	about 93%	about 58%
Functional full-design completion	about 72%	about 27%

Read the units carefully

The Ralph loop is much faster in raw iteration count, but the units are smaller. The full drain produces larger reviewed design-gap bundles and reaches much deeper into the full Workflow Lisp design.

Source: static inspection of run state, summaries, ledgers, artifacts, source, fixtures, and tests. No tests were run for this comparison.

Generating prompt: comparison metrics prompt.

Implementation quality comparison

Metric	Full Lisp drain	Ralph loop
Full implementation quality	7.8 / 10	5.8 / 10
Equivalent subset quality	8.0 / 10	6.8 / 10
Spec adherence	8.1 / 10	5.1 / 10
Correctness evidence	7.6 / 10	6.1 / 10
Test coverage	8.7 / 10	6.8 / 10
Maintainability	6.7 / 10	6.5 / 10
Churn severity	8.5 / 10	6.5 / 10

Quality readout

The full drain is heavier and churnier, but it has broader runtime integration, stronger artifact surfaces, deeper test coverage, and more durable completion evidence. The Ralph loop is smaller and fast-moving, but its subset remains less complete and less ledger-backed.

Scores are static-inspection estimates, not test results. Higher quality scores are better; higher churn severity means more repeated repair/rework signal.

Progress quality: speed vs. churn

Full Lisp drain

- Slower but more complete.
- Covers parser, typing, lowering, macros, modules, procedures, stdlib, build artifacts, diagnostics, and migration pieces.
- Hottest implementation file appears in 9 execution reports; hottest test appears in 4 reports.
- File-repeat evidence is a report-mention proxy; this run has no per-iteration changed-file ledger.

Ralph loop

- Faster early iteration velocity.
- Best suited to small MVP slices and focused regression additions.
- Changed-file ledgers show concentrated repeat touches across lowering, expression validation, and related tests.
- Hot spots: `test_workflow_lisp_lowering.py` 55 touches; `lowering.py` 52 touches.

Repeated-file disparity

Ralph's hottest source file concentration remains much higher than the full-drain report-mention proxy: 52 changed-file touches versus 9 execution-report mentions. This points to fast local iteration, but also to repeated stabilization in the same subset.

Concept: adjudicated provider

Current YAML surface

```yaml
- name: DraftDocument
  adjudicated_provider:
    candidates:
      - id: candidate_a
        provider: demo_candidate_a
      - id: candidate_b
        provider: demo_candidate_b
    evaluator:
      provider: demo_evaluator
      input_file: prompts/evaluate_candidate.md
      evidence_confidentiality: same_trust_boundary
    selection:
      tie_break: candidate_order
    score_ledger_path: artifacts/evaluations/scores.jsonl
  expected_outputs:
    - name: selected_document
      type: relpath
```

Conceptual Lisp surface

This form would live inside a surrounding `defworkflow` or `defproc`; it is not itself a top-level definition.

```
(adjudicated-provider draft-document
  :candidates
    ((candidate-a :provider providers.candidate-a)
     (candidate-b :provider providers.candidate-b))
  :evaluator providers.evaluator
  :evidence same-trust-boundary
  :tie-break candidate-order
  :returns SelectedDocument)
```

Runtime meaning

One baseline -> candidate workspaces -> output validation -> evaluator scores -> selected output promotion -> score ledger.

YAML exposes the protocol. A Lisp frontend can name the operation once and lower to the same validated candidate/evaluator/promotion machinery.

This is an early DSL-semantic step toward LLM-as-a-judge workflows: once judging is a typed operation, later search loops can evolve candidates and use adjudication as the fitness signal.

Toward self-hosted adjudication

Current v2.11 semantics

- The evaluator is a provider template plus evaluator prompt/rubric source.
- It returns strict score JSON: candidate id, finite score, and summary.
- It is not itself a nested workflow, `call`, `defproc`, or general DSL expression.

Self-hosted direction

- Treat the judge as a typed DSL unit: evidence packet in, score decision out.
- Candidate generation, judging, promotion, and score-ledger writing then live in one semantic layer.
- Genetic or search-style loops can evolve candidates while adjudication supplies the fitness function.

This keeps today's contract honest while showing the path from provider-level adjudication to workflow-native LLM-as-a-judge semantics.